State Matters: Detecting Behavioral Patterns in Object-Centric Process Mining

Dina Kretzschmann *, Alessandro Berti *, Wil M.P. van der Aalst

Posted Date: 16 July 2026

doi: 10.20944/preprints202607.1158.v1

Keywords: object-centric process mining; state-aware object-centric process mining; behavioral pattern detection; diagnostic process analysis

 doi:10.20944/preprints202607.1158.v1

Article

State Matters: Detecting Behavioral Patterns in Object-Centric Process Mining

Dina Kretzschmann [1,*], Alessandro Berti [1,*] and Wil M.P. van der Aalst [1,2]

[1] Process and Data Science (PADS), RWTH Aachen University, Aachen, Germany
[2] Celonis, Munich, Germany
* Correspondence: dina.kretzschmann@pads.rwth-aachen.de (D.K.); a.berti@pads.rwth-aachen.de (A.B.)

Abstract

Enterprise processes involve many interacting objects whose behavior depends on operational states. Object-centric process mining with OCEL 2.0 captures interactions between objects, and state-aware object-centric process mining adds the state evolution of selected objects. Identifying recurring local behavioral patterns that contribute to entering, maintaining, or recovering from undesired states is essential for process analysis and for designing improvement measures. However, detecting these patterns currently relies on manual inspection of state-aware directly-follows graphs, which is complex and does not scale. This paper presents an automated pattern detection approach for state-aware object-centric process mining. Given a leading object type, the method segments its state evolution, represents each segment as an object-centric graph, and aggregates structurally equivalent segments into ranked patterns. The method distinguishes patterns that occur inside a state from patterns that span state changes. In a real-life case study conducted with Europe's leading pet retailer, the analysis reveals the behavioral patterns most strongly associated with understock and overstock states, providing a finer-grained diagnostic view of process behavior.

Keywords: object-centric process mining; state-aware object-centric process mining; behavioral pattern detection; diagnostic process analysis

1. Introduction

Process mining derives process knowledge from event data recorded by information systems. It supports process discovery, conformance checking, performance analysis, and improvement. Most classical techniques are *case-centric*: each event is assigned to one process instance, which is convenient but often too restrictive in real-world settings.

Many processes involve several interacting business objects, such as orders, order items, deliveries, invoices, materials, patients, or machines. Events may refer to several objects, and objects may split, merge, or synchronize over time; forcing such data into a single-case view can hide dependencies and introduce convergence and divergence problems. *Object-centric process mining* (*OCPM*) with OCEL 2.0 addresses this by analyzing events together with the objects they relate to, without flattening the data [1,2].

The state information represents an operational condition of a selected object that evolves over time and is defined with respect to a process objective, thus adding another diagnostic layer. For example, in inventory management, analysts may ask which activities occur while an item-location pair is in an *understock state*, which receipt patterns create an *overstock state*, and how *overstock* is resolved. Similar questions occur in healthcare (*risk states*, *treatment phases*, *deterioration* and *recovery episodes*), in manufacturing (*setup*, *downtime*, *quality-hold*, or *rework states* of machines and production orders), and in logistics and service management, where shipments, assets, and tickets move between *normal*, *delayed*, *blocked*, and *restored states*. The approach is useful when object-centric event data is available, at least one object type has a meaningful state that changes over time, and local behavior inside states or around transitions explains cost, delay, risk, quality, or service outcomes.

 doi:10.20944/preprints202607.1158.v1

State-aware object-centric process mining (*SA-OCPM*) makes object states and state transitions explicit [3] (Figure 1), enabling joint analysis of control flow, object interactions, and state evolution. However, the resulting view becomes difficult to inspect when several states and object types are shown together, and it does not directly reveal which recurring behaviors are associated with entering an undesired state, staying in it, or recovering from it. Examples of such patterns include *partial replenishments* that fail to restore normal inventory levels, *repeated monitoring without escalation* for a patient at risk, or *repeated diagnostics and repair attempts* that prolong machine downtime. In practice, detecting these patterns requires manually inspecting state-aware directly-follows graphs, which is time-consuming, hard to reproduce, and not scalable.

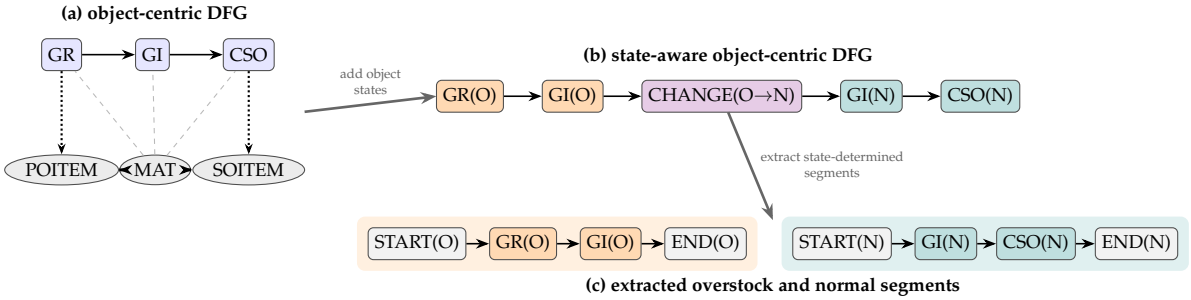


Figure 1. From an object-centric directly-follows graph to a state-aware object-centric directly-follows graph with extracted state-determined segments. The state-aware view annotates activities with the state of the leading object type; example overstock and normal segments are highlighted.

Existing aggregation mechanisms only partly address this need. *Object-centric variants* and *supervariants* summarize complete or near-complete executions and support end-to-end comparison [4], but many state-aware questions are local: analysts need to understand what repeatedly happens within a state episode of a selected object type, or around a transition from one state to another. *Object-centric local process models* capture recurring local behavior in object-centric event logs, but do not explicitly focus on state-conditioned segments of a leading object [5]. This leaves a gap for state-aware local analysis.

This paper proposes a method for detecting and ranking *state-aware object-centric behavioral patterns*. Starting from a selected *leading object type*, we derive state-determined segments, represent them as local graphs enriched with object-interaction context, and group segments with the same structure into recurring patterns. We distinguish *intra-state patterns*, which summarize behavior while the leading object remains in a state, and *inter-state patterns*, which summarize behavior around a transition between states. The result is a focused diagnostic view on how inefficient states are maintained, entered, and resolved. Figure 2 summarizes the pipeline.

The rest of the paper is organized as follows. section 2 reviews related work, section 3 introduces the concepts and data requirements, section 4 presents the approach, section 5 describes the tool support, section 6 reports the real-life case study, section 7 discusses the added value of the detected patterns relative to aggregate activity and state statistics, and section 8 concludes the paper.

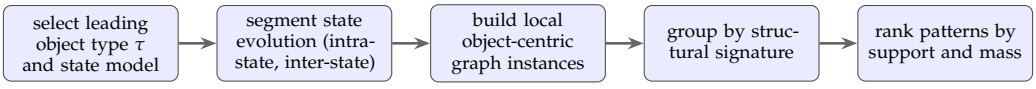


Figure 2. Overview of the method: the state evolution of a selected leading object type is segmented, transformed into local state-aware object-centric graph instances, grouped into recurring intra-state and inter-state patterns, and ranked for analysis.

2. Related Work

2.1. Object-Centric and State-Aware Process Mining

Object-centric process mining addresses the divergence and convergence problems caused by forcing event data with several object types into a single case notion [1]. Later work established core

 doi:10.20944/preprints202607.1158.v1

abstractions such as object-centric event logs, directly-follows graphs, and Petri nets [2]. OCEL 2.0 then standardized event-to-object and object-to-object relations, including evolving object attributes [6]. SA-OCPM builds on this foundation by making the states and transitions of selected objects explicit through state transition events and state-aware events [3]. Our work extends this representation by detecting recurring behaviors associated with states and state transitions.

2.2. Object-Centric Control-Flow and Diagnostic Analysis

Object-centric executions and variants support behavioral comparison without flattening event data [7], while super variants aggregate related variants for broader control-flow comparison [4]. Object-centric directly-follows graphs support conformance and performance analysis [8]. Attribute-based drill-down and roll-up can expose finer variants through Markov clustering [9], and association rules can connect object-centric behavior to process outcomes [10]. These methods mainly examine behavior across an object lifecycle, so they can merge distinct local episodes and provide limited explanations for a specific state or transition.

Object-Centric Local Process Models are closest to our work: they mine frequent object-centric behavior and represent it as object-centric Petri nets [5], but they discover local behavior in general rather than patterns associated with specific operational situations of interest. Our method instead analyzes state-determined segments of a selected leading object type, separates patterns by state episode or state transition, and retains object-interaction context. By linking pattern discovery to operational states, the resulting patterns provide diagnostic insights into recurring behavior that contributes to state persistence, state entry, and state recovery.

2.3. Data-Aware Process Mining and Decision Analysis

Data-aware process mining uses event attributes to explain process behavior, routing decisions, and outcomes [11]. Existing methods derive explicit decision rules from attribute values [12], analyze how data dependencies shape execution paths [13,14], and model how attributes influence the probability of activity execution [15]. DOCEL supports this analysis by storing dynamic object attributes over time [16].

Declarative and structural approaches use SAT solving or first-order logic to verify predefined multi-perspective behavioral templates [17,18]; these methods are top-down and require experts to specify the patterns in advance. SA-OCPM instead maps dynamic attributes to operational states in the event log [3], and our bottom-up layer automatically discovers frequent interactions and anomalies within states and across state transitions. The state definitions may rely on predefined local metrics, but the behavioral patterns need not be specified beforehand.

2.4. State-Based Modeling

State-based abstractions have been modeled through finite state machines [19], interactive lifecycle discovery [20], domain-specific approaches in healthcare [21] and manufacturing [22], and frameworks that transform raw data into higher-level process views [23]. Ontology-driven approaches often use case-centric, predefined representations in which states correspond to completed activities or temporal partitions [18], while methods for continuous sensor data commonly cluster temporal signals into newly discovered activities [24].

SA-OCPM differs by defining domain-specific operational object states that are aligned with process objectives and inferred from time-dependent attributes [3,25]. Such states have supported causal analysis of inefficient stock configurations in supply chains [26], but recurring behavioral patterns within and across states have not been explicitly addressed. Existing segmentation methods also partition event data using structural boundaries or predefined time intervals [27], without directly supporting automated intra-state and inter-state pattern analysis. Our method fills this gap by detecting frequent patterns and anomalies within states and across state transitions.

 doi:10.20944/preprints202607.1158.v1

3. Preliminaries

We build on object-centric event data [2] and on the state-aware perspective of SA-OCPM [3]. Intuitively, an object-centric event log records events, each with an activity, a timestamp, and links to one or more typed objects (e.g., materials, purchase-order items, sales-order items), so that an event affecting several objects at once is preserved as such; Table 1 shows an excerpt.

Definition 1 (Object-Centric Event Log)**.** *Let $\mathbb{T}$ be a totally ordered time domain. An* object-centric event log *is a tuple $L = (E, O, OT, A, \pi_{act}, \pi_{time}, \pi_{ot}, \pi_{omap}, \preceq)$, where E, O, OT, and A are finite sets of events, objects, object types, and activity labels; $\pi_{act} : E \to A$, $\pi_{time} : E \to \mathbb{T}$, $\pi_{ot} : O \to OT$, and $\pi_{omap} : E \to \mathcal{P}(O)$ assign to each event an activity, a timestamp, and a set of related objects, and to each object a type; and $\preceq$ is a total order on E consistent with timestamps.*

For $\tau \in OT$, let $O_\tau := \{o \in O \mid \pi_{ot}(o) = \tau\}$. The *projection* $\text{proj}(L, o) = \langle e_1, \ldots, e_n \rangle$ of L on an object o contains exactly the events related to o, ordered by $\preceq$. We use the event-to-object relation $R_{EO} := \{(e, o) \mid o \in \pi_{omap}(e)\}$ and the induced object-to-object relation $R_{OO} := \{(o_1, o_2) \mid o_1 \neq o_2,\ \exists e \in E :\ (e, o_1), (e, o_2) \in R_{EO}\}$.

A *leading object type* $\tau \in OT$ is the type whose state evolution structures the analysis; its choice is a modeling decision and should correspond to objects whose state has operational meaning. All other types are *non-leading*; they remain attached to events and later become part of the pattern context, e.g., purchase- and sales-order items help explain why a material entered, stayed in, or left an inventory state.

Definition 2 (State Model and State-Aware Labels)**.** *Fix a leading object type $\tau \in OT$. A* state model *for τ consists of a finite set of states S_τ and a function $\sigma_\tau : O_\tau \times \mathbb{T} \to S_\tau$ assigning a state to each object of type τ at each point in time. For $o \in O_\tau$ and $e \in E$ with $o \in \pi_{omap}(e)$, we abbreviate $\sigma_\tau(o, e) := \sigma_\tau(o, \pi_{time}(e))$ and define the* state-aware label *of e from the perspective of o as $\lambda_\tau(o, e) := (\pi_{act}(e), \sigma_\tau(o, e))$.*

The construction of σ_τ is domain-specific: states may be derived from thresholds, rules, predictions, clinical scores, or machine signals. In the inventory assessment, the leading type is *material* at item-location granularity and the states $\{\text{Understock}, \text{Normal}, \text{Overstock}\}$ follow from Min-Max thresholds. The label $\lambda_\tau(o, e)$ is perspective-dependent, since one event may relate to several leading objects that are simultaneously in different states.

Independently of the chosen state semantics, the state function σ_τ can be operationalized in two complementary ways. In the *expression-based* way, the analyst defines states through a rule over event- and object-level attributes that are valid at the event timestamp, as in a Min-Max comparison of a time-valid inventory level against thresholds; this is appropriate when domain knowledge already fixes the relevant operational conditions. In the *data-driven* way, a window around each event in the leading object's lifecycle is encoded as a feature vector, the encodings are clustered, and the resulting groups serve as candidate states that the analyst can inspect, label, and refine [25]; this is useful when states must be discovered from the data. Both yield a state model in the sense of Definition 2 and can be combined, e.g., clustering proposes states that are then formalized as an expression.

The state evolution of a leading object is split into *state-determined segments*: maximal episodes in one state, and pairs of neighboring episodes around a state change.

 doi:10.20944/preprints202607.1158.v1

Table 1. Event-level excerpt from the OCEL for a single item-location pair $i = (m, \ell)$, showing the minimal object links and attributes needed for the proposed approach. The state-aware activity label appends the inventory regime transition induced by the event using thresholds $\theta_i^{\text{under}} = 91$ and $\theta_i^{\text{over}} = 150$. The last column indicates whether a row is used for demand (D), inventory update (I), supply (S), boundary context (B), or replenishment planning (R). Available at https://zenodo.org/records/13347782.

t_e	Original activity	State-aware activity	MAT	PLA	POITEM	SOITEM	Partner	q_e	$I_i(t^-)$	$I_i(t^+)$	Used for
2023-01-02 09:10	Goods Issue	Goods Issue (Normal → Normal)	M001	L01	-	SO7	C001	2	100	98	D, I
2023-01-02 12:05	Goods Issue	Goods Issue (Normal → Normal)	M001	L01	-	SO8	C002	3	98	95	D, I
2023-01-03 08:30	Create Purchase Order Item	Create Purchase Order Item (Normal)	M001	L01	PO1	-	S001	100	95	95	S, B, R
2023-01-04 10:15	Goods Issue	Goods Issue (Normal → Understock)	M001	L01	-	SO9	C003	5	95	90	D, I
2023-01-10 14:20	Goods Receipt	Goods Receipt (Understock → Normal)	M001	L01	PO1	-	S001	50	90	140	S, I
2023-01-10 14:21	Goods Receipt	Goods Receipt (Normal → Overstock)	M001	L01	PO1	-	S001	50	140	190	S, I
2023-01-11 09:40	Goods Issue	Goods Issue (Overstock → Overstock)	M001	L01	-	SO10	C001	10	190	180	D, I

 doi:10.20944/preprints202607.1158.v1

Definition 3 (State Episodes and State-Determined Segments). *Fix $\tau \in OT$ and $o \in O_\tau$ with* $\mathrm{proj}(L, o) = \langle e_1, \ldots, e_n \rangle$ *and* $s_i := \sigma_\tau(o, e_i)$. *A* state episode *of o is a maximal interval $[i, j]$ with $s_i = \cdots = s_j = s$ for some $s \in S_\tau$. It is represented by the boundary-augmented sequence*

$$\Gamma^{\mathrm{intra}}(o, i, j) := \langle \mathsf{START}_s, \lambda_\tau(o, e_i), \ldots, \lambda_\tau(o, e_j), \mathsf{END}_s \rangle.$$

If $[i, j]$ and $[j+1, k]$ are consecutive episodes with states $s \neq s'$, the corresponding transition episode *is represented by*

$$\Gamma^{\mathrm{inter}}(o, i, j, k) := \langle \mathsf{START}_s, \lambda_\tau(o, e_i), \ldots, \lambda_\tau(o, e_j), \mathsf{CHANGE}_{s \to s'}, \lambda_\tau(o, e_{j+1}), \ldots, \lambda_\tau(o, e_k), \mathsf{END}_{s'} \rangle.$$

Both Γ^{intra} and Γ^{inter} are called state-determined segments.

The symbols START_s, END_s, and $\mathsf{CHANGE}_{s \to s'}$ only make segment boundaries visible; they do not replace original events. For a segment Γ of a leading object o, let $\mathrm{ev}(\Gamma) \subseteq E$ be its underlying events. The segment-restricted relations $R^\Gamma_{EO} := \{(e, o') \in R_{EO} \mid e \in \mathrm{ev}(\Gamma),\ o' \neq o\}$ and $R^\Gamma_{OO} := \{(o, o') \mid \exists e \in \mathrm{ev}(\Gamma) : (e, o') \in R^\Gamma_{EO}\}$ capture the non-leading participations within Γ and the induced leading-to-non-leading interactions. Finally, $\mathcal{S}^{\mathrm{intra}}_\tau$ and $\mathcal{S}^{\mathrm{inter}}_\tau$ denote the sets of all intra-state and inter-state segments of the leading type τ.

4. Approach

Fix a leading object type $\tau \in OT$ and a state model for it. The method operates on individual state-determined segments rather than on an already aggregated directly-follows graph: it preserves the event order of each leading object and the object links that explain local behavior, transforms each segment into a local typed graph enriched with object interactions, and groups structurally identical instances into ranked patterns. The result can still be visualized as local weighted subgraphs of a state-aware object-centric directly-follows graph; Figure 7 illustrates this correspondence on the case study of section 6.

Pattern families. Let $s, s' \in S_\tau$ with $s \neq s'$. We distinguish

$$\mathcal{S}^{\mathrm{intra}}_{\tau,s} := \{\Gamma \in \mathcal{S}^{\mathrm{intra}}_\tau \mid \Gamma \text{ starts with } \mathsf{START}_s\}$$

and

$$\mathcal{S}^{\mathrm{inter}}_{\tau,s \to s'} := \{\Gamma \in \mathcal{S}^{\mathrm{inter}}_\tau \mid \mathsf{CHANGE}_{s \to s'} \text{ occurs in } \Gamma\}.$$

These families answer different diagnostic questions: what repeatedly happens *while* an object remains in state s, and what repeatedly happens *around* a crossing from s to s', e.g., entry into overstock or recovery back to normal.

Local graphs. Each segment becomes a local graph with control-flow nodes for its state-aware labels and boundary markers, object-type nodes for the participating non-leading types, and three kinds of weighted edges: directly-follows edges record the order of state-aware activities along the leading object, event-to-object-type edges record which non-leading types participate in which activities, and object-to-object-type edges record which non-leading types are connected to the leading object inside the segment.

Definition 4 (Segment Pattern Instance). *Let Γ be a state-determined segment of a leading object $o_\Gamma \in O_\tau$. The* segment pattern instance *of Γ is the typed weighted graph $H_\tau(\Gamma) = (V^\Gamma_C, V^\Gamma_O, F^\Gamma_{DF}, F^\Gamma_{EO}, F^\Gamma_{OO}, \omega^\Gamma_{DF}, \omega^\Gamma_{EO}, \omega^\Gamma_{OO})$ with*

- $V^\Gamma_C := \{\gamma \mid \gamma \text{ occurs in } \Gamma\}$, *the control-flow nodes (state-aware labels and boundary symbols), and* $V^\Gamma_O := \{\tau\} \cup \{\pi_{ot}(o') \mid (e, o') \in R^\Gamma_{EO}\}$, *the participating object types;*
- $\omega^\Gamma_{DF}(x, y) := \#\{r \mid \Gamma[r] = x,\ \Gamma[r+1] = y\}$ *for $x, y \in V^\Gamma_C$;*
- $\omega^\Gamma_{EO}(x, \tau') := |\{(e, o') \in R^\Gamma_{EO} \mid \lambda_\tau(o_\Gamma, e) = x,\ \pi_{ot}(o') = \tau'\}|$ *for $x \in V^\Gamma_C$ and $\tau' \in V^\Gamma_O \setminus \{\tau\}$;*

 doi:10.20944/preprints202607.1158.v1

- $\omega^{\Gamma}_{OO}(\tau, \tau') := |\{o' \mid (o_\Gamma, o') \in R^{\Gamma}_{OO},\ \pi_{ot}(o') = \tau'\}|$ *for* $\tau' \in V^{\Gamma}_{O} \setminus \{\tau\}$;

and edge sets F^{Γ}_{DF}, F^{Γ}_{EO}, F^{Γ}_{OO} *containing exactly the pairs with positive weight.*

This representation keeps the segment local while remaining object-centric: a pattern can show that an overstock episode is driven by receipts and repeated issues, and whether purchase- or sales-order items participate.

Structural abstraction. To obtain reusable patterns, we abstract from concrete object identifiers and retain only the labeled structure; Figure 3 shows two instances that differ in weights but share the same structure.

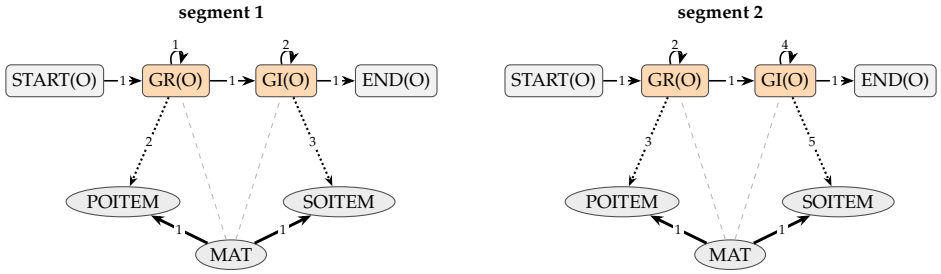


Figure 3. Two segments with the same structural signature but different weights. Arc styles denote directly-follows, leading-type, event-to-object-type, and object-to-object-type relations.

Definition 5 (Structural Signature)**.** *The* structural signature *of a segment pattern instance is*

$$\mathrm{sig}(H_\tau(\Gamma)) := (V^{\Gamma}_{C}, V^{\Gamma}_{O}, F^{\Gamma}_{DF}, F^{\Gamma}_{EO}, F^{\Gamma}_{OO}).$$

Two instances are structurally equivalent *if they have the same signature.*

Since nodes are already labeled by state-aware activity labels, boundary symbols, and object types, structural equivalence coincides with label-preserving isomorphism; weights are excluded from the signature and used only for aggregation and ranking.

Definition 6 (Behavioral Pattern)**.** *Fix a family* $\mathcal{S} \in \{\mathcal{S}^{\mathrm{intra}}_{\tau,s}, \mathcal{S}^{\mathrm{inter}}_{\tau,s\to s'}\}$. *A* behavioral pattern *is an equivalence class* $P = [\Gamma]_{\equiv} := \{\Gamma' \in \mathcal{S} \mid \mathrm{sig}(H_\tau(\Gamma')) = \mathrm{sig}(H_\tau(\Gamma))\}$. *Its* representative graph $H(P)$ *consists of the common signature of all members of P together with aggregated weights obtained by summing componentwise over the instances, e.g.,* $W^{P}_{DF}(x,y) := \sum_{\Gamma\in P} \omega^{\Gamma}_{DF}(x,y)$, *and analogously* W^{P}_{EO} *and* W^{P}_{OO}.

A behavioral pattern is thus a recurring segment structure together with the accumulated evidence for it. Patterns from $\mathcal{S}^{\mathrm{intra}}_{\tau,s}$ are *intra-state patterns*; patterns from $\mathcal{S}^{\mathrm{inter}}_{\tau,s\to s'}$ are *inter-state patterns*. Figure 4 shows an inter-state pattern for an overstock-to-normal transition. **Ranking.** Patterns are ranked primarily by *support* $|P|$, the number of represented segment instances, with ties broken by the *control-flow mass* $\mathrm{mass}(P) := \sum_{(x,y)\in F^{P}_{DF}} W^{P}_{DF}(x,y)$, where F^{P}_{DF} is the directly-follows edge set of the common signature. This separates common routine behavior from rarer but process-heavy behavior. Other statistics, such as duration or monetary impact, can be attached after discovery without affecting pattern identity.

Detection procedure. For a fixed leading type τ, detection selects a target family of state-determined segments, constructs $H_\tau(\Gamma)$ for each segment, groups identical signatures into behavioral patterns, aggregates their weights, and ranks the patterns by support and mass. The procedure applies to any state or ordered state transition; the assessment analyzes intra-state patterns for Understock and Overstock and inter-state patterns for Normal $\to$ Overstock and Overstock $\to$ Normal.

 doi:10.20944/preprints202607.1158.v1

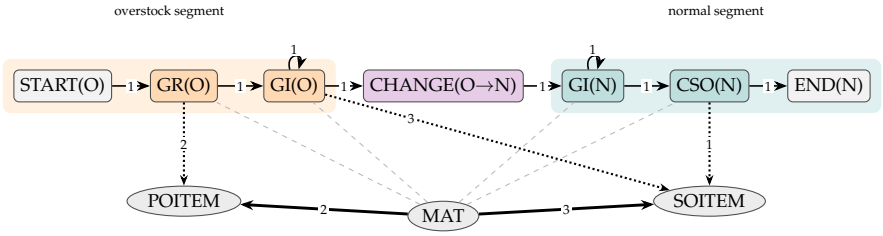
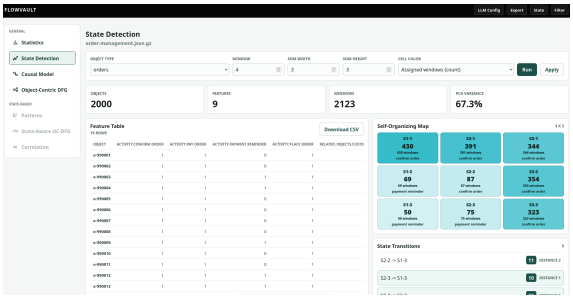


Figure 4. Inter-state pattern for an overstock-to-normal transition. The overstock and normal segments are highlighted in different colors. Arc styles denote directly-follows, leading-type, event-to-object-type, and object-to-object-type relations.

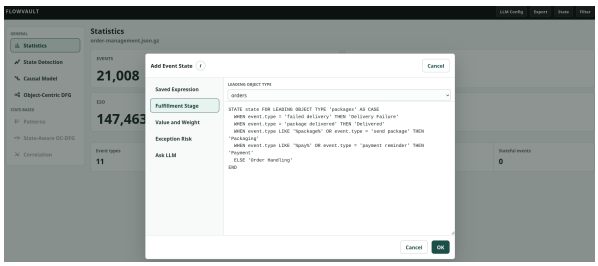
5. Tool Support

The approach is implemented in *Flowvault*, a browser-based environment for OCEL 2.0 event data. Flowvault imports JSON and XML logs (also gzip-compressed), runs its core analyses in Rust/WebAssembly, and provides filtering, statistics, state enrichment and detection, object-centric directly-follows graphs, and the pattern analysis from section 4. The repository is available at https://github.com/fit-alessandro-berti/flowvault and a public deployment at https://www.flowvault.cloud/index.html.

States can be established in two complementary ways (Figure 5): analysts may select a leading object type and define states through a SQL-like `CASE` expression over events and time-valid object attributes, or Flowvault derives lifecycle-window features and maps them, via principal components, to a self-organizing map whose cells provide candidate states [25].



(a) Automatic state definition



(b) Manual state definition

Figure 5. State definition in Flowvault. (a) Automatic state detection using lifecycle-window features, principal component analysis, and a self-organizing map [25]. (b) Manual event-state enrichment through a SQL-like expression for a selected leading object type.

After enrichment, Flowvault segments leading-object lifecycles into intra-state and inter-state behavior, groups structurally equal local object-centric graphs, and ranks the patterns by support and control-flow mass. The interface supports frequency-ranked selection, text and graph views, and filtering to matching behavior (Figure 6).

 doi:10.20944/preprints202607.1158.v1

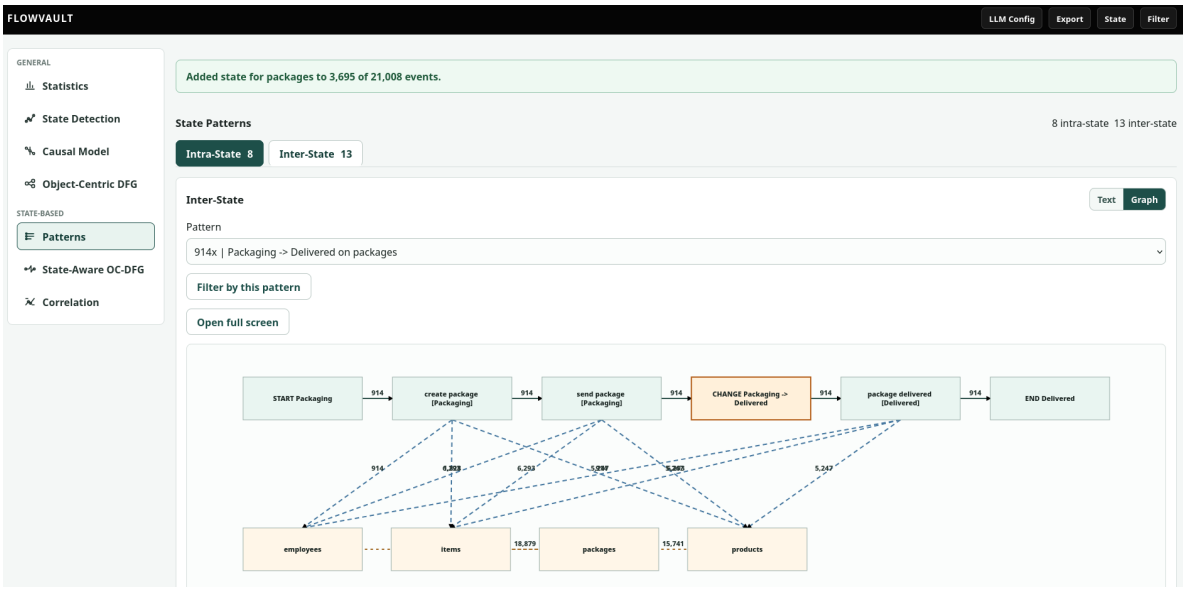


Figure 6. State-aware pattern exploration in Flowvault, including intra-state and inter-state families, frequency-ranked selection, graph visualization, and filtering by the selected pattern.

It is important to note that Flowvault was tested on multiple object-centric event logs from different scenarios, such as order management and logistics, indicating that the approach and tool support generalize beyond the inventory setting.

6. Case Study

We assess the method on a real-life inventory management object-centric event log of Europe's leading pet retail company, which generated €3.66 billion in revenue in 2025, employs more than 20,000 people, operates 10 warehouses, and offers pet products from several hundred suppliers in more than 2,880 stores and online. The implementation is available at https://github.com/fit-alessandro-berti/causal-model-inventory-management; the reported patterns were computed with `state_aware_pattern_detection.py`, which groups leading inventory objects by item-location, segments contiguous state periods, mines recurring structures within and across states, and outputs JSON and CSV files for reproducibility. The data is a 5% sample of the retailer's inventory log with leading objects of type MAT at item-location granularity: 2,183,919 event rows, 614 leading material objects, and four activity classes (`Create Purchase Order Item`, `Create Sales Order Item`, `Goods Issue`, and `Goods Receipt`), covering 2023-01-02 to 2024-03-01 (424 days).

In Table 2, **CPO**, **CSO**, **GI**, and **GR** abbreviate the four activities and **N**, **O**, and **U** the three states; **START** and **END** denote episode boundaries and **CHANGE** a transition. **Support** counts matching segments, **Mass** aggregates directly-follows transitions, and **Avg. trans.** is their average per match. **Family share** is calculated within the segment family and is not comparable to the log-level shares in Table 3. Panels (a) and (b) cover contiguous Overstock and Understock episodes, (c) and (d) the two transitions.

Overstock episodes (Table 2a, Figure 7) are heterogeneous: many are short and driven by physical inventory movements, with receipt followed by outbound issues dominating, while a smaller set shows repeated sales-order and purchase-order handling continuing while the pair is already overstocked. **Understock episodes** (Table 2b) are more concentrated: *Goods Issue* drives the state, procurement or sales-order handling follows, and goods receipt is rarely visible within the same episode, so recovery is often initiated but completed later. Understock is thus a demand-pressure state, less process-heavy than overstock, but corrective actions may arrive too late to prevent shortage.

 doi:10.20944/preprints202607.1158.v1

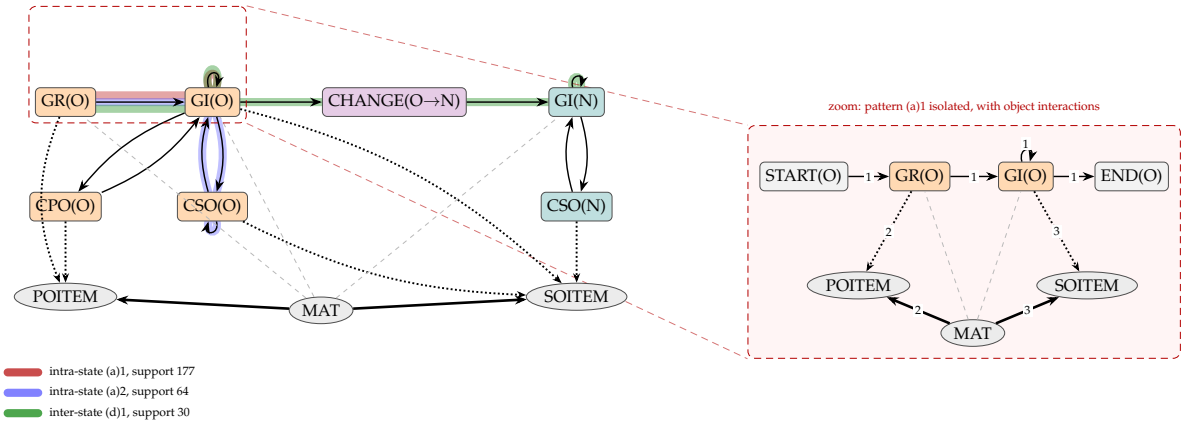


Figure 7. Projection of detected patterns onto an excerpt of the state-aware object-centric directly-follows graph. Two intra-state overstock patterns (ranks (a)1 and (a)2 in Table 2, red and blue) and one inter-state Overstock → Normal pattern (rank (d)1, green) are highlighted along the arcs they traverse; shared arcs, such as GR(O) → GI(O), carry several projections. The zoom isolates pattern (a)1 together with its interactions with purchase-order items (POITEM) and sales-order items (SOITEM) of the leading material object type (MAT): each detected pattern is a local weighted subgraph of the state-aware graph.

Normal → Overstock transitions (Table 2c) show a clear tendency: *Goods Issue* activity on the normal side, a crossing associated with a *Goods Receipt*, and continued *Goods Issue* afterwards. Overstock is rarely created by an isolated event; it emerges when an inbound receipt enters an active operational context in which purchase- and sales-order handling may continue across the boundary. **Overstock → Normal transitions** (Table 2d, Figure 7) are diverse but recognizable: the overstock side is shaped by *Goods Receipt* and repeated *Goods Issue*, and the normal side continues with further *Goods Issue*. Normalization is achieved through depletion of excess stock rather than a corrective posting and, since planning and coordination often persist across the boundary, is better read as recovery followed by stabilization.

Improvement. For overstock, receipt quantities and timing should be reviewed for materials that enter overstock repeatedly, replenishment decisions taken at high stock should be checked against projected inventory, and a monitoring view can flag repeat entries and process-heavy episodes. For understock, improvement should focus on earlier replenishment triggers, monitoring materials approaching the threshold without an open replenishment action, and tracking procurement execution after shortage detection; repeated sales-order handling may indicate allocation issues. To prevent Normal → Overstock transitions, open replenishment quantities should be checked against projected inventory before posting; order timing and cancellation discipline reduce excess stock. Recovery from overstock should be managed as a post-normalization control phase, checking replenishment decisions for recently normalized materials and reviewing conflicting interventions in process-heavy transitions.

 doi:10.20944/preprints202607.1158.v1

Table 2. Top six patterns per segment family: (a) overstock episodes, (b) understock episodes, (c) Normal → Overstock transitions, (d) Overstock → Normal transitions.

Rank	Simplified pattern	Support	Family share	Mass	Avg. trans.	Business interpretation
(a) Overstock episode patterns						
1	START → GR → GI → GI → END	177	18.6%	708	4.0	Short routine overstock episode. Stock is received and reduced through outbound issues.
2	GR and GI with repeated GI ↔ CSO alternation and CSO self-loop	64	6.7%	1218	19.0	Overstock is worked down while sales-order handling is adjusted repeatedly.
3	GR → GI, then CPO, then return to GI before exit	42	4.4%	290	6.9	Procurement enters the overstock episode, but the pattern remains contained.
4	GR, GI, CSO, and CPO all present, with strong looping around GI and CSO	27	2.8%	750	27.8	Sales and procurement replanning occur during overstock.
5	GR and GI with a shorter GI ↔ CSO alternation	22	2.3%	172	7.8	Sales-order handling is involved, but the loop is shorter than in pattern (a)2.
6	Heavy four-activity loop with GI, CSO, CPO, and repeated returns to GR	17	1.8%	5920	348.2	Rare but process-heavy episode. Few cases absorb much coordination effort.
(b) Understock episode patterns						
1	START → GI → GI → END	96	19.4%	288	3.0	Short shortage episode. Stock keeps being consumed while already understocked.
2	START → GI → GI → CPO → END	34	6.9%	136	4.0	Continued depletion is followed by a procurement response.
3	START → CPO → CPO → END	31	6.3%	93	3.0	Planning-driven understock episode with repeated purchase-order activity.
4	START → GI → GI → CSO → CSO → END	27	5.5%	135	5.0	Demand-side handling becomes visible during shortage.
5	GI with repeated branching to CPO and return to GI before exit	22	4.4%	133	6.0	Procurement has started, but the shortage continues.
6	START → GI → GI → CSO → END	21	4.2%	84	4.0	Sales-order handling is involved, but the episode remains short.
(c) Normal → Overstock transition patterns						
1	GI(N) self-loop → CHANGE → GR(O) → GI(O) self-loop	24	3.5%	216	9.0	Normal-side depletion is followed by a receipt that creates overstock. Issues continue afterward.
2	GI(N) self-loop → CHANGE → GR(O), then GI(O) ↔ CPO(O)	18	2.6%	230	12.8	The transition is simple before the boundary but procurement-heavy after overstock is reached.
3	GI(N) ↔ CPO(N) → CHANGE → GR(O) → GI(O)	17	2.5%	220	12.9	Procurement is active while stock is normal, but the receipt still pushes the item into overstock.
4	GI(N) → CPO(N) → CHANGE → GR(O) → GI(O)	14	2.0%	140	10.0	Planning in the normal state is followed by a procurement-driven transition.
5	GI(N) self-loop → CHANGE → GR(O), with GI(O) ↔ CPO(O) and repeated returns to GR(O)	9	1.3%	216	24.0	After the boundary, overstock handling involves receipt, issue, and procurement interaction.
6	GI(N) ↔ CSO(N) → CHANGE → GR(O) → GI(O) ↔ CSO(O)	7	1.0%	350	50.0	Sales-side coordination persists across the state boundary.
(d) Overstock → Normal transition patterns						
1	GR(O) → GI(O) self-loop → CHANGE → GI(N) continuation	30	4.7%	267	8.9	Canonical normalization pattern. Excess stock is worked down through outbound issues.
2	GR(O) → GI(O) self-loop → CHANGE → GI(N) ↔ CPO(N)	21	3.3%	277	13.2	After normalization, procurement planning feeds back into normal-state execution.
3	GR(O), then GI(O) ↔ CPO(O) → CHANGE → GI(N)	18	2.8%	227	12.6	Procurement is present before normalization, while the normal side is simpler.
4	GR(O), then GI(O) ↔ CSO(O) → CHANGE → GI(N) ↔ CSO(N)	13	2.0%	728	56.0	Sales-order handling persists across the boundary and creates heavier cases.
5	GR(O) → GI(O) self-loop → CHANGE → GI(N) → CPO(N)	13	2.0%	129	9.9	Normalization is followed by a short procurement action.
6	GR(O), with mixed CSO(O) and CPO(O) activity before CHANGE, then GI(N) ↔ CSO(N)	8	1.3%	363	45.4	Planning actions are active before and after normalization.

7. Discussion

A natural question is whether the detected patterns add value beyond the aggregate activity and state counts in Table 3, which show a process dominated by *Goods Issue* and the overstock state, with

 doi:10.20944/preprints202607.1158.v1

understock below one percent of the log. Such counts report how often activity-state combinations occur, but not how events are ordered, whether a receipt precedes or follows a state change, or whether behavior forms short episodes or repeated loops. In isolation, they can mislead: frequent *Create Purchase Order Item (Overstock)* might suggest unnecessary overstock, yet the patterns show these actions are often followed by *Goods Issue (Overstock)*, i.e., procurement embedded in short demand-driven episodes.

Table 3. Activity and state counts in the inventory log.

Activity	State	Count	Share of log
Goods Issue	Overstock	1,419,805	65.01%
Goods Issue	Normal	562,898	25.77%
Create Sales Order Item	Overstock	100,124	4.58%
Create Sales Order Item	Normal	43,665	2.00%
Goods Receipt	Overstock	12,286	0.56%
Goods Issue	Understock	12,045	0.55%
Create Purchase Order Item	Overstock	11,132	0.51%
Create Purchase Order Item	Normal	8,181	0.37%
Goods Receipt	Normal	8,063	0.37%
Create Sales Order Item	Understock	5,096	0.23%
Create Purchase Order Item	Understock	558	0.03%
Goods Receipt	Understock	66	$< 0.01\%$
	Total	2,183,919	100.00%

The detected patterns supply this missing structure: intra-state overstock patterns expose sales- and procurement-heavy loops that absorb effort, inter-state patterns show rare *Goods Receipt* events are central around Normal $\rightarrow$ Overstock transitions, and understock patterns reveal depletion whose corrective actions often remain incomplete.

This added value rests on three properties: the method separates intra-state persistence from inter-state transitions, retains interactions with related object types, and distinguishes routine from rare high-mass behavior. These properties complement SA-OCPM by extending state visibility towards automated discovery of recurring state-dependent behavior.

8. Conclusion

This paper introduced a method for detecting and ranking state-aware object-centric behavioral patterns: it extracts state-determined segments of a selected leading object type and aggregates structurally equivalent local graphs into recurring intra-state and inter-state patterns. Thus, the method complements SA-OCPM with a diagnostic view on how states are *maintained*, *entered*, and *resolved*. The approach is implemented in Flowvault and evaluated in a real-world inventory case study, demonstrating that the discovered patterns provide concise diagnostic evidence and that state transitions are often part of broader coordination behavior rather than isolated events. The application of the approach involves design choices regarding the selection of the leading object type, the definition of states, and the ranking criteria. Future work will investigate richer ranking criteria, interactive visualizations, and combinations with predictive and prescriptive techniques.

Acknowledgments: Funded by the European Union. This work has received funding from the European High Performance Computing Joint Undertaking (JU) and from the German Federal Ministry of Research, Technology and Space (BMFTR), the Ministry of Culture and Science of North Rhine-Westphalia (MKW NRW), and the Hessian Ministry of Science and Research, Arts and Culture (HMWK) under grant agreement No 101250682.

References

1. van der Aalst, W. Object-Centric Process Mining: Dealing with Divergence and Convergence in Event Data. In Proceedings of the SEFM. Springer, 2019, Vol. 11724, *Lecture Notes in Computer Science*, pp. 3–25.

 doi:10.20944/preprints202607.1158.v1

2. van der Aalst, W. Object-Centric Process Mining: Unraveling the Fabric of Real Processes. *Mathematics* **2023**, *11*.
3. Kretzschmann, D.; Berti, A.; van der Aalst, W.M.P. State-Aware Object-Centric Process Mining: Enhancing OCEL 2.0 with Explicit State Transitions. In Proceedings of the EDOC. Springer, 2025, Lecture Notes in Computer Science, pp. 103–118.
4. Adams, J.N.; Hastrup-Kiil, E.; Park, G.; van der Aalst, W.M.P. Super Variants. In Proceedings of the BPM. Springer, 2024, Lecture Notes in Computer Science, pp. 111–128.
5. Peeva, V.; Porsil, M.; van der Aalst, W.M.P. Object-Centric Local Process Models. In Proceedings of the ICPM Workshops. Springer, 2024, Lecture Notes in Business Information Processing, pp. 376–388.
6. Koren, I.; Adams, J.N.; Berti, A.; van der Aalst, W.M.P. OCEL 2.0 Resources - www.ocel-standard.org. *CoRR* **2024**, *abs/2403.01982*.
7. Adams, J.N.; Schuster, D.; Schmitz, S.; Schuh, G.; van der Aalst, W.M.P. Defining Cases and Variants for Object-Centric Event Data. In Proceedings of the ICPM. IEEE, 2022, pp. 128–135.
8. Park, G.; Adams, J.N.; van der Aalst, W.M.P. Conformance Checking and Performance Analysis Using Object-Centric Directly-Follows Graphs. In Proceedings of the BPM (Forum). Springer, 2024, Lecture Notes in Business Information Processing, pp. 179–196.
9. Miri, N.; Jalali, A. Uncovering patterns in object-centric process mining: an approach using drill-down and roll-up techniques. In Proceedings of the International conference on information integration and web intelligence. Springer, 2024, pp. 49–54.
10. Knopp, B.; Pourbafrani, M.; van der Aalst, W.M.P. Root Cause Analysis Using Rule Mining on Object-Centric Event Logs. In Proceedings of the ICPM Workshops. Springer, 2024, Lecture Notes in Business Information Processing, pp. 57–69.
11. Piccirilli, E.; Di Ciccio, C.; Montali, M.; Peñaloza, R.; Pontieri, L.; Ricca, F. Explainable Knowledge-Aware Process Intelligence: PINPOINT Final Project Report. *KI-Künstliche Intelligenz* **2025**, *39*, 311–316.
12. De Leoni, M.; van der Aalst, W.M.P. Data-aware process mining: discovering decisions in processes using alignments. In Proceedings of the Proc. ACM Symp. Appl. Comput., 2013, pp. 1454–1461.
13. Rifki, O.; Peng, Z.; Perrier, L.; Xie, X. Process mining with event attributes and transition features for care pathway modelling. *International Journal of Production Research* **2025**, *63*, 3684–3708.
14. Nadim, K.; Ragab, A.; Ouali, M.S. Data-driven dynamic causality analysis of industrial systems using interpretable machine learning and process mining. *J. Intell. Manuf.* **2023**, *34*, 57–83.
15. Mannhardt, F.; Leemans, S.J.; Schwanen, C.T.; de Leoni, M. Modelling data-aware stochastic processes-discovery and conformance checking. In Proceedings of the International Conference on Applications and Theory of Petri Nets and Concurrency. Springer, 2023, pp. 77–98.
16. Goossens, A.; De Smedt, J.; Vanthienen, J.; van der Aalst, W.M.P. Enhancing data-awareness of object-centric event logs. In Proceedings of the ICPM. Springer, 2022, pp. 18–30.
17. Maggi, F.M.; Marrella, A.; Patrizi, F.; Skydanienko, V. Data-aware declarative process mining with SAT. *ACM Trans. Intell. Syst. Technol.* **2023**, *14*, 1–26.
18. Moher, R.; Gruninger, M. Mining for Meaning: Ontology-Aware Process Mining Methods Through Knowledge Patterns. In Proceedings of the International Conference on Research Challenges in Information Science. Springer, 2025, pp. 109–119.
19. Hompes, B.F.A.; van der Aalst, W.M.P. Lifecycle-based process performance analysis. In Proceedings of the CoopIS, C&TC, ODBASE. Springer, 2018, pp. 336–353.
20. van Eck, M.L.; Sidorova, N.; van der Aalst, W.M.P. Discovering and exploring state-based models for multi-perspective processes. In Proceedings of the BPM. Springer, 2016, pp. 142–157.
21. Valero-Ramon, Z.; Fernandez-Llatas, C.; Valdivieso, B.; Traver, V. Dynamic models supporting personalised chronic disease management through healthcare sensors with interactive process mining. *Sensors* **2020**, *20*, 5330.
22. Celik, U.; Yurtay, Y. Improvement of assemble-to-order model processes with process mining: dynamic analysis and hybrid approaches. *IEEE Access* **2025**.
23. Diba, K.; Batoulis, K.; Weidlich, M.; Weske, M. Extraction, correlation, and abstraction of event data for process mining. *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* **2020**, *10*, e1346.
24. Ziolkowski, T.; Koschmider, A.; Schubert, R.; Renz, M. Process mining for time series data. In Proceedings of the Enterprise, Business-Process and Information Systems Modeling: 23rd International Conference, BPMDS 2022 and 27th International Conference, EMMSAD, 2022, pp. 6–7.

 doi:10.20944/preprints202607.1158.v1

25. Berti, A.; Kretzschmann, D.; van der Aalst, W.M. Interpretable Execution State Abstraction from Object-Centric Event Logs for Process-Aware Decision Support: Linking Execution States to Stock and Policy Regimes. In Proceedings of the International Conference on Advanced Information Systems Engineering. Springer, 2026, pp. 321–333.
26. Berti, A.; Kretzschmann, D.; Aalst, W.M.V.D. From Object-Centric Event Data to Causal Inventory Insights: A Structural Equation Model of Understock and Overstock **2026**.
27. Berti, A.; Kretzschmann, D.; van der Aalst, W.M.P. Segmentation for Optimizing Long-Lifecycle Processes in Object-Centric Process Mining. **2025**.